

From Skill Guidance to Verified Execution: Auditing Financial Research Agents

Working draft: author details pending confirmation

September 21, 2026

Draft under revision; not submission-ready.

Abstract

Financial research agents can produce executable code and plausible performance reports without establishing that the underlying research is valid. We develop an audit workflow that connects source-dated domain guidance, executable checks, and evidence bound to the final submitted artifact. The workflow separates a failed check from an incomplete audit and distinguishes executing a guard from merely citing it. An independent evaluator perturbs market inputs and recomputes portfolio performance. A historical exploratory pilot of 12 runs motivates a four-condition study of guidance, optional tools, and mandatory final checks. In the pilot, the library-associated change in reporting error differs across model tiers; these observations do not establish a general benefit or a causal mechanism. We describe the implemented protocol, its synthetic regression evidence, and the additional experiments and data required before a capability claim.

1 Research question

An agent’s statement that it checked a backtest is not evidence that the check ran, used the relevant inputs, or applied to the final code. Even a genuine passing check only addresses its declared scope. We ask whether binding audit evidence to final research artifacts reduces incorrect accepted conclusions beyond textual guidance and optional tools. This question concerns the reliability of research outputs, rather than whether an agent discovers a profitable trading strategy.

The proposed contribution has three parts: a financial knowledge and check interface; a submission protocol that records actual execution and audit completeness; and an independent evaluation of accepted and rejected outputs. The current implementation supports this investigation. Its efficacy across models remains an empirical question.

2 Related work

CheckList formalizes behavioral testing beyond aggregate task accuracy [5]. Financial backtest overfitting and selection bias predate LLM agents [2, 1]. Profit Mirage investigates information leakage in financial agents and proposes counterfactual interventions [4]. The Finance Agent Benchmark evaluates real financial research tasks [3]. Thus neither financial-agent leakage nor counterfactual testing alone is a new contribution here. Our intended distinction is the connection between artifact-bound execution evidence, accepted-output risk, and controlled comparisons of textual guidance with optional and mandatory checks. Establishing that distinction requires the study below, not a feature-count comparison.

3 System and audit contract

Knowledge and checks. The repository contains source-dated skill documents and Python guards with structured findings. Skills describe library conventions and research failure modes. Guards inspect supplied artifacts through a common interface. A verification date records a source check; it does not imply that every statement remains current indefinitely.

Audit completeness. A task specifies a nonempty set of required guards. A report is `PASS` only when all required checks ran and passed with no rejected input; a detected defect yields `FAIL`, while missing or rejected checks yield `INCOMPLETE` in the absence of a detected failure. Warnings remain visible. A passing subset is not a certificate that the whole research task is valid.

Execution evidence. The controller runs checks on a snapshot of the actual submission and supplied data. It records each result, elapsed time, and a SHA-256 digest of the code, input files, manifest and numerical report claims. Changing those artifacts invalidates earlier receipts. Citation-list changes alone do not invalidate a check. Unknown guard names and execution errors cannot create successful execution records. Executed-but-failed, unexecuted, and stale checks are distinct states.

The current intervention binds the library’s close-price causality and survivorship guards to submitted code and positions. It additionally checks reported accounting and samples same-session dependencies. These checks cover a subset of possible defects. The runner exposes a fixed guidance excerpt, not the entire skill collection.

4 Independent evaluation

A seeded synthetic market provides quoted prices, corporate actions, listings, post-close commentary, pre-computed scores and fundamental disclosures. A submitted function returns portfolio weights; a separate report states its net Sharpe ratio. The task now specifies that session- t weights must use information available by the end of session $t - 1$, consistent with close-to-close return accounting.

The independent evaluator redraws future data and individual sessions, measures changes in submitted positions, checks post-delisting exposure, and recomputes performance at a fixed transaction-cost convention. Public intervention checks do not call this oracle. Private grades are generated only after the submission receipt is saved and are not returned to the agent for repair. Rejected submissions are also graded when executable.

The oracle’s fixed-cost Sharpe and the public tool’s stated-cost Sharpe answer different questions. Their discrepancy can reflect accounting conventions as well

Historical arm	Mean absolute Sharpe gap
Opus, no library	0.188
Opus, library	0.034
Haiku, no library	0.574
Haiku, library	11.949

Table 1: Exploratory pilot: three runs per arm. Values come from saved grades; model versions and full interaction-level execution provenance require further verification.

as coding errors. Neither should be called an estimate of genuine alpha without timing and other validity checks. A latent-drift strategy’s sample Sharpe is a reference observation, not a theoretical upper bound. Exceeding it is not mathematical proof of leakage.

5 Evidence available so far

5.1 Synthetic regression checks

Saved robustness results record 36 detections among 36 planted instances: twelve defect families repeated across three seeded worlds, with no reported clean-control false alarms in those worlds. This is regression evidence within one synthetic design. Repeating a seed design does not supply independently authored defects or estimate a general real-world false-positive rate. The detection suite exercises thirteen guards, not every registered guard.

5.2 Historical agent pilot

The saved pilot comprises three markets, two model tiers, and library-present versus library-absent conditions, with one run per cell. Table 1 recomputes the mean absolute discrepancy from the saved raw grades. The observed relative reduction for Opus is 81.9%, computed before rounding. It is not a significance claim.

The Haiku library arm has the largest discrepancy in this pilot. This motivates testing whether guidance, tool use, or other differences explain the outcome. It does not show that documentation causes weaker models to fail or that mandatory checks solve the problem. Historical reports of unexecuted guard citations require original execution logs for verification; the saved grade table alone does not establish that behavior.

5.3 Numerical and real-data checks

The numerical benchmark compares implementations with reference calculations using case-specific tolerances. Passing within tolerance is not exact equality. Runtime measurements report the names and number of guards actually exercised, skipped and rejected checks, and machine-dependent timing. A fitted slope across a few data sizes is descriptive, not an asymptotic complexity bound.

The KOL fixture supports recomputing profile-group aggregates. Its model prediction metrics are saved summaries, not values independently recomputed from predictions and outcomes. In particular, its stored pooled Rank IC and mean daily cross-sectional Rank IC can have different signs. We do not treat this fixture as end-to-end reproduction, evidence of profitable trading, or external validation of the agent intervention. A separate evaluator now requires timestamped row-level predictions and outcomes, reports both forms of IC, and uses paired date-block uncertainty. Those source records remain an outstanding evidence requirement.

6 Four-condition study

The new protocol compares task guidance with ordinary accounting feedback, the same setup plus a fixed skill excerpt, the excerpt plus optional executable guards, and the excerpt plus a mandatory final audit. Market data, inference seeds and budgets are paired. Every condition has the same turn and response-token caps; actual token use, execution time and tool calls are also recorded. Conditions run in a frozen randomized order. Failed requests have no hidden retry or synthetic response fallback.

The primary outcome is incorrect accepted submissions divided by all attempted submissions. Correctness among accepted outputs and overall acceptance rate must be reported alongside it, so refusing every output cannot appear to solve the task. Secondary outcomes include independent leakage measures, reporting discrepancy, post-delisting exposure, unexecuted and stale citations, repair success and resource cost. Missing, rejected, ungradable and provider-failed runs remain separate outcomes.

The first Beacon run is a public-seed feasibility study using pinned open-weight models. It tests operation and supports cost and sample-size planning.

A confirmatory study additionally needs independent defect authorship, retained clean cases, repeated runs, and verified scorer isolation. Market-level paired uncertainty must preserve repetitions within markets. The present protocol is an amendment after examining the pilot, not a preregistration of its observations. Failure to find a return difference would not establish equivalence.

Operational findings. The first Beacon preflight rejected valid tool-call JSON wrapped in Markdown fences. We archived those records and amended the parser before a separate batch; this was an interface failure, not evidence of financial reasoning failure. In the revised batch, Qwen2.5-Coder 7B and 14B each completed 12 recorded cells, with 0 and 7 accepted submissions respectively. Neither model produced an independent numerical grade: final code failed input or output validation, or a required artifact was absent. Accepted does not mean correct, and these observations cannot estimate improvement in financial integrity.

The 32B run encountered nonfinite generation values and a CUDA error before a tool call. Subsequent failure records from that process are not independent capability observations. A separately recorded single-GPU recovery run remains queued. A handwritten causal control passed the mandatory audit and independent timing/accounting checks on one public market. That control verifies an executable path, not general scorer validity. Before a confirmatory study, basic artifact validity must be distinguished from the incremental contribution of financial checks.

Limitations and remaining work

The Beacon worker uses Linux Landlock and seccomp to restrict filesystem access, network calls and process creation; unsupported enforcement fails closed. Local subprocess execution alone provides no such isolation. Submitted Python still shares an interpreter with individual checks, so host confinement does not establish that the evaluator resists tampering. A formal adversarial or private-holdout claim needs an independently protected scorer and additional validation. Publicly inspected generator code and seeds are development data. Finite perturbations can miss conditional dependencies; stochastic strategies need repeatability controls. A passing audit is limited to its input assump-

tions and required checks. The full multi-model study and row-level real-world reproduction are not yet established.

Authorship, data and disclosure status

Author order, affiliations, funding, data permissions and final declarations require confirmation by the accountable authors. The earlier draft’s unconfirmed organization and email details have been removed. AI assistance was used for code review, revisions and experiment preparation in this work session; the final disclosure and human review record must be completed before submission. The manuscript remains a working draft.

References

- [1] David H Bailey, Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69, 2017.
- [2] David H Bailey and Marcos López de Prado. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.
- [3] Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. Finance agent benchmark: Benchmarking llms on real-world financial research tasks. *arXiv preprint arXiv:2508.00828*, 2025.
- [4] Xiangyu Li, Yawen Zeng, Xiaofen Xing, Jin Xu, and Xiangmin Xu. Profit mirage: Revisiting information leakage in llm-based financial agents. *arXiv preprint arXiv:2510.07920*, 2025.
- [5] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912, 2020.